

Universidad Autónoma del Estado de México

Facultad de Ingeniería

Unidad 3

Algoritmos de Búsqueda y Ordenamiento

Conceptos básicos y ejemplos en Python

Contenido temático

Algoritmos de búsqueda (lineal y binaria)

Algoritmos de ordenamiento (secuencial, selección, inserción, burbuja, quicksort y mergesort)

Aplicaciones prácticas con herramientas de manejo de datos

Introducción

Los algoritmos de búsqueda y ordenamiento son piezas fundamentales. Casi cualquier aplicación de software realiza, en algún momento de su ejecución, operaciones de localización de datos o de reorganización de información para presentarla de manera útil al usuario. Comprender cómo funcionan estos algoritmos, en qué condiciones se comportan mejor y cuál es su costo en términos de tiempo y memoria permite tomar decisiones informadas al diseñar soluciones de software.

Este documento aborda los temas de la Unidad 3 del programa: algoritmos de búsqueda (lineal y binaria), algoritmos de ordenamiento (secuencial, selección, inserción, burbuja, quicksort y mergesort) y aplicaciones prácticas con herramientas de manejo de datos. Cada algoritmo se presenta con su definición, una explicación paso a paso, un ejemplo implementado en Python y una discusión breve de su complejidad. Al final se incluyen ejercicios para reforzar el aprendizaje.

Notación de complejidad — O grande

La notación $O(\dots)$ describe cómo crece el tiempo de ejecución (o el uso de memoria) en función del tamaño de la entrada n . $O(1)$ significa tiempo constante, $O(\log n)$ logarítmico, $O(n)$ lineal, $O(n \log n)$ linealítmico y $O(n^2)$ cuadrático. A mayor crecimiento, peor desempeño cuando n es grande.

3.1 Algoritmos de búsqueda

Un algoritmo de búsqueda es un procedimiento que, dada una colección de elementos y un valor objetivo (llamado clave), determina si la clave está presente en la colección y, en caso afirmativo, devuelve su posición. Existen muchas variantes según la estructura de los datos y las condiciones del problema; aquí se estudian las dos más representativas: búsqueda lineal y búsqueda binaria.

3.1.1 Búsqueda lineal (secuencial)

La búsqueda lineal recorre la colección elemento por elemento, desde el inicio hasta el final, comparando cada uno con la clave buscada. Si encuentra una coincidencia, devuelve la posición; si llega al final sin encontrarla, devuelve un valor que indica ausencia (típicamente -1).

Características clave: no requiere que los datos estén ordenados, es muy fácil de implementar y funciona con cualquier tipo de colección iterable. Su desventaja es que en el peor caso debe revisar todos los elementos.

Ejemplo en Python

```
def busqueda_lineal(lista, clave):
    """
    Busca 'clave' en 'lista' recorriendo elemento por elemento.
    Devuelve el índice si la encuentra, -1 si no.
    """
    for indice in range(len(lista)):
        if lista[indice] == clave:
            return indice
    return -1

# Ejemplo de uso
numeros = [14, 7, 3, 25, 9, 41, 18]
resultado = busqueda_lineal(numeros, 25)

if resultado != -1:
    print(f"El valor 25 está en la posición {resultado}")
else:
    print("El valor 25 no se encuentra en la lista")

# Salida:
# El valor 25 está en la posición 3
```

Análisis

- **Mejor caso:** $O(1)$ — el elemento buscado es el primero.

- **Caso promedio:** $O(n)$ — en promedio se revisa la mitad de la lista.
- **Peor caso:** $O(n)$ — el elemento está al final o no existe.
- **Memoria adicional:** $O(1)$.

3.1.2 Búsqueda binaria

La búsqueda binaria es un algoritmo mucho más eficiente, pero exige una condición indispensable: la colección debe estar previamente ordenada. La estrategia consiste en dividir repetidamente el rango de búsqueda a la mitad, descartando en cada paso la mitad donde la clave no puede estar.

En cada iteración se calcula el índice central del rango actual y se compara su valor con la clave: si son iguales, la búsqueda termina; si la clave es menor, se continúa en la mitad izquierda; si es mayor, en la mitad derecha. Este proceso reduce el espacio de búsqueda a la mitad cada vez, lo que produce una complejidad logarítmica.

Requisito fundamental

La búsqueda binaria SOLO funciona sobre colecciones ordenadas. Si los datos no lo están, debe ordenarse primero (lo cual tiene su propio costo). Para una sola búsqueda en datos no ordenados, la búsqueda lineal suele ser más conveniente; la binaria conviene cuando se harán muchas búsquedas sobre los mismos datos.

Ejemplo en Python — versión iterativa

```
def busqueda_binaria(lista, clave):
    """
    Búsqueda binaria iterativa.
    Precondición: 'lista' debe estar ordenada ascendentemente.
    """
    izquierda = 0
    derecha = len(lista) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2

        if lista[medio] == clave:
            return medio
        elif lista[medio] < clave:
            izquierda = medio + 1 # buscar en la mitad derecha
        else:
            derecha = medio - 1   # buscar en la mitad izquierda

    return -1
```

```
# Ejemplo
ordenados = [3, 7, 9, 14, 18, 25, 41]
print(búsqueda_binaria(ordenados, 18)) # Salida: 4
print(búsqueda_binaria(ordenados, 10)) # Salida: -1
```

Ejemplo en Python — versión recursiva

```
def búsqueda_binaria_recursiva(lista, clave, izq, der):
    if izq > der:
        return -1

    medio = (izq + der) // 2

    if lista[medio] == clave:
        return medio
    elif lista[medio] < clave:
        return búsqueda_binaria_recursiva(lista, clave, medio + 1, der)
    else:
        return búsqueda_binaria_recursiva(lista, clave, izq, medio - 1)

# Uso
datos = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(búsqueda_binaria_recursiva(datos, 23, 0, len(datos) - 1)) # 5
```

Análisis

- **Mejor caso:** $O(1)$ — la clave está justo en el centro.
- **Caso promedio:** $O(\log n)$.
- **Peor caso:** $O(\log n)$ — se necesitan $\lceil \log_2 n \rceil$ comparaciones.
- **Memoria adicional:** $O(1)$ iterativa, $O(\log n)$ recursiva por la pila.

3.1.3 Comparación entre búsqueda lineal y binaria

Para visualizar la diferencia, consideremos una lista con un millón de elementos ($n = 1\,000\,000$). La búsqueda lineal puede requerir hasta un millón de comparaciones, mientras que la búsqueda binaria necesitará a lo más 20 comparaciones ($\log_2 1\,000\,000 \approx 19.93$). Esta diferencia se vuelve crítica cuando los datos crecen.

3.2 Algoritmos de ordenamiento

Ordenar es reorganizar los elementos de una colección siguiendo un criterio (numérico, alfabético, por fecha, etc.). El ordenamiento es una operación tan común y tan importante que se ha estudiado extensivamente; existen decenas de algoritmos, cada uno con ventajas particulares. En esta sección se presentan seis: secuencial, selección, inserción, burbuja, quicksort y mergesort.

Algoritmos in-place y estables

Un algoritmo es 'in-place' si ordena usando memoria adicional constante (no requiere copiar la lista). Es 'estable' si conserva el orden relativo de elementos con la misma clave, lo cual es importante cuando se ordena por múltiples criterios (por ejemplo, alumnos por apellido manteniendo el orden por nombre previo).

3.2.1 Ordenamiento secuencial

El ordenamiento secuencial es la forma más directa de ordenar: se recorre la lista posición por posición, y para cada posición se asegura que contenga el valor correcto buscando el mínimo (o el máximo) en lo que queda por procesar. En la práctica suele identificarse con el ordenamiento por selección, pero conceptualmente puede entenderse como cualquier estrategia que procese los elementos en secuencia para colocarlos en su lugar definitivo, uno a la vez.

Ejemplo en Python — variante didáctica

```
def ordenamiento_secuencial(lista):
    """
    Recorre la lista posición por posición y coloca en cada
    una el valor que le corresponde, comparando con el resto.
    """
    n = len(lista)
    for i in range(n - 1):
        for j in range(i + 1, n):
            if lista[j] < lista[i]:
                lista[i], lista[j] = lista[j], lista[i]
    return lista

numeros = [29, 10, 14, 37, 13]
print(ordenamiento_secuencial(numeros))
# Salida: [10, 13, 14, 29, 37]
```

Análisis

- **Complejidad:** $O(n^2)$ en todos los casos.

- **Memoria:** $O(1)$ — in-place.
- **Uso recomendado:** fines didácticos y listas muy pequeñas.

3.2.2 Ordenamiento por selección (Selection Sort)

La idea es sencilla: en cada pasada, se busca el elemento más pequeño del subarreglo no ordenado y se intercambia con el primer elemento de ese subarreglo. Tras cada pasada, el prefijo ordenado crece en un elemento. Es muy intuitivo y minimiza el número de intercambios (a lo más $n - 1$), aunque hace muchas comparaciones.

Algoritmo paso a paso

- Asume que el primer elemento es el mínimo.
- Recorre el resto y, si encuentra uno menor, actualiza la posición del mínimo.
- Al terminar el recorrido, intercambia el mínimo con el elemento de la posición actual.
- Repite con la siguiente posición, hasta llegar al penúltimo elemento.

Ejemplo en Python

```
def seleccion(lista):
    n = len(lista)
    for i in range(n - 1):
        # Encontrar el índice del mínimo en lista[i..n-1]
        indice_min = i
        for j in range(i + 1, n):
            if lista[j] < lista[indice_min]:
                indice_min = j

        # Intercambiar si el mínimo no está ya en su lugar
        if indice_min != i:
            lista[i], lista[indice_min] = lista[indice_min], lista[i]

    return lista

datos = [64, 25, 12, 22, 11]
print(seleccion(datos))
# Salida: [11, 12, 22, 25, 64]
```

Análisis

- **Mejor / promedio / peor caso:** $O(n^2)$.
- **Memoria:** $O(1)$.

- **Estable:** no (en su versión clásica).

3.2.3 Ordenamiento por inserción (Insertion Sort)

Es similar a la forma en que muchas personas ordenan cartas en la mano: se toma el siguiente elemento y se inserta en la posición correcta dentro de la parte ya ordenada, desplazando los elementos mayores una posición a la derecha. Es muy eficiente cuando la lista está casi ordenada o cuando se trabaja con conjuntos pequeños.

Algoritmo paso a paso

- Comienza desde el segundo elemento (índice 1).
- Guarda el valor actual en una variable auxiliar (clave).
- Compara la clave con los elementos anteriores; mientras sean mayores, los desplaza una posición a la derecha.
- Coloca la clave en el hueco resultante.
- Repite con cada elemento hasta el final.

Ejemplo en Python

```
def insercion(lista):
    for i in range(1, len(lista)):
        clave = lista[i]
        j = i - 1

        # Desplaza los elementos mayores que 'clave' hacia la derecha
        while j >= 0 and lista[j] > clave:
            lista[j + 1] = lista[j]
            j -= 1

        # Inserta 'clave' en su posición correcta
        lista[j + 1] = clave

    return lista

numeros = [12, 11, 13, 5, 6]
print(insercion(numeros))
# Salida: [5, 6, 11, 12, 13]
```

Análisis

- **Mejor caso:** $O(n)$ — lista ya ordenada.

- **Caso promedio y peor:** $O(n^2)$.
- **Memoria:** $O(1)$.
- **Estable:** sí.

3.2.4 Ordenamiento de burbuja (Bubble Sort)

Su nombre proviene del modo en que los elementos más grandes 'burbujean' hacia el final de la lista en cada pasada. En cada iteración se comparan pares de elementos adyacentes y, si están en el orden incorrecto, se intercambian. Después de $n - 1$ pasadas la lista queda ordenada. Es el algoritmo más conocido por su simplicidad, pero también uno de los más ineficientes.

Versión optimizada con bandera

Se incluye una variable booleana que detecta si en una pasada no hubo intercambios; si no los hubo, la lista ya está ordenada y se puede terminar antes.

Ejemplo en Python

```
def burbuja(lista):
    n = len(lista)
    for i in range(n - 1):
        hubo_intercambio = False

        # En cada pasada el mayor sube al final del rango activo
        for j in range(n - 1 - i):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                hubo_intercambio = True

        # Si no hubo intercambios, la lista ya está ordenada
        if not hubo_intercambio:
            break

    return lista

datos = [5, 1, 4, 2, 8]
print(burbuja(datos))
# Salida: [1, 2, 4, 5, 8]
```

Análisis

- **Mejor caso:** $O(n)$ con la optimización de bandera.
- **Caso promedio y peor:** $O(n^2)$.

- **Memoria:** $O(1)$.
- **Estable:** sí.

3.2.5 Quicksort

Quicksort, propuesto por Tony Hoare en 1960, es uno de los algoritmos de ordenamiento más utilizados en la práctica gracias a su excelente rendimiento promedio. Sigue la estrategia 'divide y vencerás': se elige un elemento llamado pivote, se reorganiza la lista de manera que los menores queden a la izquierda y los mayores a la derecha, y luego se aplica recursivamente el mismo procedimiento a ambas mitades.

Algoritmo paso a paso

- Si la lista tiene 0 o 1 elementos, ya está ordenada.
- Selecciona un pivote (por simplicidad puede ser el último elemento).
- Particiona la lista en dos: menores o iguales al pivote y mayores que el pivote.
- Aplica quicksort recursivamente a cada partición.
- Concatena: izquierda + pivote + derecha.

Ejemplo en Python — versión clara con listas auxiliares

```
def quicksort(lista):
    if len(lista) <= 1:
        return lista

    pivote = lista[len(lista) // 2]  # pivote = elemento del medio
    menores = [x for x in lista if x < pivote]
    iguales = [x for x in lista if x == pivote]
    mayores = [x for x in lista if x > pivote]

    return quicksort(menores) + iguales + quicksort(mayores)

datos = [3, 6, 8, 10, 1, 2, 1, 7, 4]
print(quicksort(datos))
# Salida: [1, 1, 2, 3, 4, 6, 7, 8, 10]
```

Ejemplo en Python — versión in-place (esquema de Lomuto)

```
def quicksort_inplace(lista, izq=0, der=None):
    if der is None:
        der = len(lista) - 1

    if izq < der:
        # Particiona y obtén la posición final del pivote
        p = particion(lista, izq, der)
        quicksort_inplace(lista, izq, p - 1)
        quicksort_inplace(lista, p + 1, der)
```

```
    return lista

def particion(lista, izq, der):
    pivote = lista[der]
    i = izq - 1

    for j in range(izq, der):
        if lista[j] <= pivote:
            i += 1
            lista[i], lista[j] = lista[j], lista[i]

    lista[i + 1], lista[der] = lista[der], lista[i + 1]
    return i + 1

datos = [10, 7, 8, 9, 1, 5]
print(quicksort_inplace(datos))
# Salida: [1, 5, 7, 8, 9, 10]
```

Análisis

- **Mejor caso y promedio:** $O(n \log n)$.
- **Peor caso:** $O(n^2)$ — ocurre con pivotes mal elegidos (lista ya ordenada y pivote en los extremos).
- **Memoria:** $O(\log n)$ por la recursión.
- **Estable:** no en su forma clásica.

Buena práctica

Para evitar el peor caso, en implementaciones reales se elige el pivote como la mediana de tres (primer, medio y último elemento), o aleatoriamente. Esto hace que el peor caso sea altamente improbable en datos reales.

3.2.6 Mergesort

Mergesort, ideado por John von Neumann en 1945, también aplica la estrategia 'divide y vencerás', pero con un enfoque distinto: divide la lista a la mitad recursivamente hasta obtener sublistas de un solo elemento (que ya están ordenadas por definición) y luego las fusiona ordenadamente. Garantiza complejidad $O(n \log n)$ en todos los casos, pero requiere memoria adicional proporcional al tamaño de la entrada.

Algoritmo paso a paso

- Si la lista tiene 0 o 1 elementos, devolverla directamente (caso base).
- Dividir la lista en dos mitades aproximadamente iguales.
- Aplicar mergesort recursivamente a cada mitad.
- Fusionar (merge) las dos sublistas ordenadas en una sola sublista ordenada.

Ejemplo en Python

```
def mergesort(lista):
    if len(lista) <= 1:
        return lista

    medio = len(lista) // 2
    izquierda = mergesort(lista[:medio])
    derecha = mergesort(lista[medio:])

    return fusionar(izquierda, derecha)

def fusionar(izq, der):
    """Combina dos listas ordenadas en una sola lista ordenada."""
    resultado = []
    i = j = 0

    # Avanza por las dos listas tomando el menor en cada paso
    while i < len(izq) and j < len(der):
        if izq[i] <= der[j]:
            resultado.append(izq[i])
            i += 1
        else:
            resultado.append(der[j])
            j += 1

    # Agrega los restantes (uno de los dos quedará vacío)
    resultado.extend(izq[i:])
    resultado.extend(der[j:])

    return resultado
```

```

datos = [38, 27, 43, 3, 9, 82, 10]
print(mergesort(datos))
# Salida: [3, 9, 10, 27, 38, 43, 82]

```

Análisis

- **Mejor, promedio y peor caso:** $O(n \log n)$ — muy predecible.
- **Memoria:** $O(n)$ — necesita listas auxiliares para la fusión.
- **Estable:** sí — preserva el orden relativo.

3.2.7 Tabla comparativa de algoritmos de ordenamiento

A continuación se resumen las complejidades de los algoritmos estudiados. Esta tabla es una referencia útil al momento de elegir un algoritmo según el contexto del problema:

Algoritmo	Mejor caso	Caso promedio	Peor caso	Memoria
Secuencial	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Observaciones prácticas: para conjuntos pequeños (menos de 20 a 50 elementos), inserción suele ser la mejor opción por su baja constante; para conjuntos grandes, quicksort y mergesort dominan; cuando se requiere estabilidad y desempeño garantizado, mergesort es la elección natural; cuando interesa el desempeño promedio en memoria limitada, quicksort suele preferirse.

3.3 Aplicaciones prácticas con herramientas de manejo de datos

Aunque entender la mecánica interna de los algoritmos es fundamental, en la práctica profesional se trabaja con herramientas que ya implementan estos algoritmos de manera eficiente. Python ofrece funciones nativas para buscar y ordenar, y bibliotecas como NumPy y pandas extienden estas capacidades a grandes volúmenes de datos. Esta sección muestra cómo aplicar lo aprendido usando esas herramientas.

3.3.1 Búsqueda y ordenamiento con funciones nativas de Python

Python incluye funciones que implementan algoritmos eficientes y probados. Para ordenar, `sorted()` y `list.sort()` usan internamente Timsort, un algoritmo híbrido (combinación de mergesort e inserción) con complejidad $O(n \log n)$ en el peor caso y $O(n)$ en el mejor (datos casi ordenados). Para buscar, el módulo `bisect` implementa búsqueda binaria sobre listas ordenadas.

Funciones nativas para ordenar

```
# sorted() devuelve una nueva lista ordenada
numeros = [5, 2, 8, 1, 9, 3]
ordenados = sorted(numeros)
print(ordenados)          # [1, 2, 3, 5, 8, 9]
print(numeros)            # [5, 2, 8, 1, 9, 3] (no se modifica)

# list.sort() ordena la lista en sitio (in-place)
numeros.sort()
print(numeros)            # [1, 2, 3, 5, 8, 9]

# Orden descendente con reverse=True
print(sorted(numeros, reverse=True)) # [9, 8, 5, 3, 2, 1]

# Ordenar por una clave (criterio personalizado)
alumnos = [
    {"nombre": "Ana", "calif": 8.5},
    {"nombre": "Luis", "calif": 9.2},
    {"nombre": "Eva", "calif": 7.8},
]
por_calificacion = sorted(alumnos, key=lambda a: a["calif"], reverse=True)
for a in por_calificacion:
    print(a["nombre"], a["calif"])
# Luis 9.2
# Ana 8.5
# Eva 7.8
```

Búsqueda binaria con el módulo bisect

```
import bisect
```

```

ordenados = [1, 3, 5, 7, 9, 11, 13, 15]

# bisect_left: posición donde insertar (a la izquierda de iguales)
pos = bisect.bisect_left(ordenados, 7)
print(pos)    # 3 – el 7 está en el índice 3

# Verificación de existencia
def existe(lista, clave):
    pos = bisect.bisect_left(lista, clave)
    return pos < len(lista) and lista[pos] == clave

print(existe(ordenados, 7))    # True
print(existe(ordenados, 10))   # False

# Insertar manteniendo el orden
bisect.insort(ordenados, 8)
print(ordenados)    # [1, 3, 5, 7, 8, 9, 11, 13, 15]

```

3.3.2 Aplicación con NumPy

NumPy es la biblioteca estándar para cómputo numérico en Python. Sus arreglos (ndarray) están optimizados en C y ofrecen operaciones vectorizadas mucho más rápidas que las listas nativas para grandes volúmenes de datos numéricos.

Ordenamiento y búsqueda con NumPy

```

import numpy as np

datos = np.array([23, 5, 17, 8, 42, 11, 30])

# Ordenar (usa quicksort por defecto, soporta mergesort y heapsort)
ordenados = np.sort(datos)
print(ordenados)    # [ 5  8 11 17 23 30 42]

# Especificar algoritmo
print(np.sort(datos, kind="mergesort"))

# Obtener los índices que ordenarían el arreglo
indices = np.argsort(datos)
print(indices)    # [1 3 5 2 0 6 4]

# Búsqueda binaria sobre arreglos ordenados
pos = np.searchsorted(ordenados, 17)
print(pos)    # 3 – el 17 está en el índice 3

```



```
# Buscar varios valores a la vez (vectorizado)
buscar = np.array([8, 30, 100])
print(np.searchsorted(ordenados, buscar))  # [1 5 7]
```

Por qué NumPy es rápido

Comparemos el tiempo de ordenar un millón de números con la lista nativa y con NumPy:

```
import time, random
import numpy as np

n = 1_000_000
lista = [random.random() for _ in range(n)]
arreglo = np.array(lista)

# Python nativo
inicio = time.time()
sorted(lista)
print(f"Python sorted(): {time.time() - inicio:.3f} s")

# NumPy
inicio = time.time()
np.sort(arreglo)
print(f"NumPy np.sort(): {time.time() - inicio:.3f} s")

# Típicamente NumPy es entre 3x y 10x más rápido para datos numéricos.
```

3.3.3 Aplicación con pandas

pandas es la biblioteca por excelencia para análisis de datos tabulares en Python. Permite cargar archivos CSV, Excel, bases de datos o JSON, y trabajar con ellos como tablas (DataFrames) con índices etiquetados, donde ordenar y buscar son operaciones cotidianas.

Ejemplo integrador: lista de alumnos

```
import pandas as pd

# Crear un DataFrame con datos de alumnos
alumnos = pd.DataFrame({
    "matricula": [101, 102, 103, 104, 105],
    "nombre":    ["Ana", "Luis", "Eva", "Diego", "Sofía"],
    "calif":     [8.5, 9.2, 7.8, 6.4, 9.7],
    "carrera":   ["ICO", "ICO", "ISC", "ICO", "ISC"],
})
```

```
# Ordenar por calificación descendente
top = alumnos.sort_values(by="calif", ascending=False)
print(top)
#   matricula nombre  calif carrera
# 4         105  Sofía    9.7      ISC
# 1         102   Luis    9.2      ICO
# 0         101   Ana     8.5      ICO
# 2         103   Eva     7.8      ISC
# 3         104  Diego    6.4      ICO

# Ordenar por varios criterios (carrera y luego calificación)
combinado = alumnos.sort_values(by=["carrera", "calif"], ascending=[True,
False])
print(combinado)

# Búsqueda: alumnos con calificación >= 9
aprobados = alumnos[alumnos["calif"] >= 9]
print(aprobados)

# Búsqueda exacta por nombre
fila = alumnos[alumnos["nombre"] == "Eva"]
print(fila)
```

Lectura de un archivo CSV y ordenamiento

```
import pandas as pd

# Suponga un archivo 'ventas.csv' con columnas: producto, mes, total
df = pd.read_csv("ventas.csv")

# Mostrar los 5 productos con mayor venta total
top_5 = df.sort_values(by="total", ascending=False).head(5)
print(top_5)

# Guardar el resultado ordenado
top_5.to_csv("top_ventas.csv", index=False)
```

3.3.4 Caso integrador: análisis de calificaciones

El siguiente programa integra búsqueda y ordenamiento aplicados a un problema realista. Se simula una lista de calificaciones de un grupo y se realizan varias operaciones típicas.

```
import random
```

```
import bisect

# 1. Generar 30 calificaciones aleatorias entre 5.0 y 10.0
random.seed(42)
calificaciones = [round(random.uniform(5.0, 10.0), 1) for _ in range(30)]

# 2. Ordenar usando el algoritmo nativo (Timsort,  $O(n \log n)$ )
calificaciones_ord = sorted(calificaciones)
print("Calificaciones ordenadas:")
print(calificaciones_ord)

# 3. Calcular estadísticas básicas usando la lista ordenada
minimo = calificaciones_ord[0]
maximo = calificaciones_ord[-1]
mediana = calificaciones_ord[len(calificaciones_ord) // 2]
promedio = sum(calificaciones_ord) / len(calificaciones_ord)

print(f"\nMínimo: {minimo}")
print(f"Máximo: {maximo}")
print(f"Mediana: {mediana}")
print(f"Promedio: {promedio:.2f}")

# 4. Búsqueda binaria: ¿cuántos alumnos aprobaron ( $\geq 6.0$ )?
indice_6 = bisect.bisect_left(calificaciones_ord, 6.0)
aprobados = len(calificaciones_ord) - indice_6
print(f"\nAlumnos con 6.0 o más: {aprobados}")

# 5. Top 5 calificaciones (las últimas 5 de la lista ordenada)
print(f"Top 5: {calificaciones_ord[-5:]})")
```

Ejercicios propuestos

Para reforzar lo aprendido se sugieren los siguientes ejercicios. Se recomienda resolverlos sin usar las funciones nativas de ordenamiento o búsqueda, salvo cuando el enunciado lo indique expresamente.

Nivel básico

- Implementar la búsqueda lineal de manera que, en lugar de devolver el primer índice donde se encuentra la clave, devuelva una lista con todos los índices donde aparece.
- Modificar la búsqueda binaria iterativa para que devuelva la posición de inserción cuando la clave no exista (similar a `bisect_left`).
- Implementar el ordenamiento por selección sobre una lista de cadenas, ordenando alfabéticamente sin distinguir mayúsculas y minúsculas.
- Modificar el ordenamiento de burbuja para que ordene de forma descendente.

Nivel intermedio

- Implementar quicksort eligiendo el pivote como la mediana de tres (primer, medio y último elemento) y comparar empíricamente el número de comparaciones con la versión que toma el último elemento como pivote.
- Implementar mergesort de manera iterativa (sin recursión), usando un enfoque ascendente que fusione sublistas de tamaño 1, luego 2, luego 4, etcétera.
- Dado un archivo CSV con campos de alumnos (matrícula, nombre, calificación), leerlo con pandas, ordenar por calificación descendente y guardar el resultado en un nuevo archivo.

Nivel avanzado

- Implementar una función que mida y compare empíricamente los tiempos de ejecución de los seis algoritmos vistos para listas aleatorias de tamaño 100, 1 000, 5 000 y 10 000. Presentar los resultados en una gráfica usando matplotlib.
- Diseñar una función que use búsqueda binaria para encontrar la raíz cuadrada entera de un número (mayor entero r tal que $r^2 \leq n$) sin usar la función `sqrt`.
- Investigar el funcionamiento de Timsort (algoritmo nativo de Python) y explicar en qué se diferencia de mergesort e inserción, y por qué resulta tan eficiente para datos del mundo real.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4ª ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4ª ed.). Addison-Wesley.

- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. Wiley.
- Python Software Foundation (2024). The Python Standard Library — bisect, sorted, list.sort. <https://docs.python.org/>
- NumPy Developers (2024). NumPy Reference Guide. <https://numpy.org/doc/>
- pandas Development Team (2024). pandas Documentation. <https://pandas.pydata.org/docs/>